

Manual Técnico – BattleScript

Organización de Lenguajes y Compiladores 1 – Proyecto 1

Zabdi Merari Adina Donis Rosales - 202308487

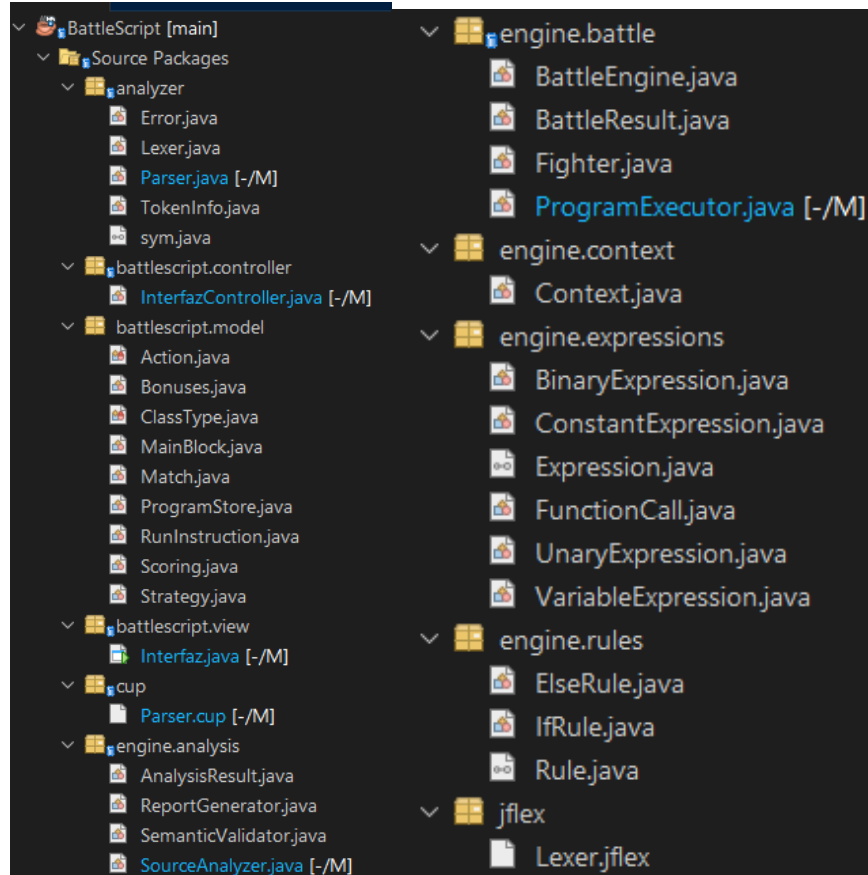
Contents

1. Descripción general	1
2. Herramientas utilizadas	1
3. Estructura del proyecto	2
4. Flujo de ejecución	2
5. Analizador léxico (<code>Lexer.jflex</code>)	2
6. Analizador sintáctico (<code>Parser.cup</code>)	3
7. Modelo del lenguaje (<code>battlescript.model</code>)	3
8. Expresiones, reglas y contexto	3
9. Motor de batalla (<code>engine.battle</code>)	4
10. Análisis y reportes (<code>engine.analysis</code>)	5
11. Interfaz gráfica	5
12. Compilación y ejecución	5
13. Mantenimiento futuro	5

1. Descripción general

BattleScript es un intérprete de un lenguaje declarativo propio (`.bt1`) para definir estrategias automáticas de combate por turnos entre magos y guerreros. Implementa un **analizador léxico** (JFlex), un **analizador sintáctico** (CUP), un **motor de ejecución de batallas** y una **interfaz gráfica** (Java Swing).

2. Estructura del proyecto



3. Flujo de ejecución

```
Interfaz (editor) → InterfazController.run()
    → SourceAnalyzer.analyze(código)
        1) Lexer → tokens / errores léxicos
        2) Parser → ProgramStore / errores sintácticos
        3) SemanticValidator → errores semánticos
    → showTokens() / showErrors() / ReportGenerator (reporte HTML)
    → (si no hay errores) ProgramExecutor.execute(programa)
        → BattleEngine.run(match, jugador1, jugador2, seed)
    → resultados mostrados en la interfaz
```

4. Analizador léxico (Lexer.jflex)

Generado con JFlex; produce `Symbol` para CUP. Configuración: `%class Lexer,%cup,%line %column` (para reportar línea/columna) y un estado extra `%state COMMENT` para comentarios multilinea.

Tokens principales: palabras reservadas de estructura (`mage`, `warrior`, `initial`, `rules`, `if`, `then`, `else`, `match`, `players`, `rounds`, `scoring`, `bonuses`, `main`, `run`, `with`, `seed`);

estados del sistema (self_health, opponent_health, self_resource, opponent_resource, self_score, opponent_score, round_number, total_rounds, self_history, opponent_history, random);

propiedades de puntuación/combo (damage_point, healing_point, successful_defense, victory_bonus, failed_action_penalty, mage_combo(_points), warrior_combo(_points), low_health_victory);

funciones (get_moves_count, get_last_n_moves, get_move, last_move);

acciones de mago → MAGE_ACTION (ARCANE_BOLT, FIREBALL, MAGIC_BARRIER, HEALING_RUNE, MEDITATE);

acciones de guerrero → WARRIOR_ACTION (SLASH, HEAVY_STRIKE, SHIELD_BLOCK, WAR_CRY, REST);

operadores (== != >= <= > <, && || !);

símbolos { } [] () : , ;

literales ENTERO, DECIMAL, e ID (identificador).

Comentarios `//... y /*...*/` se ignoran (no generan token); un comentario multilínea sin cerrar antes de EOF, o cualquier carácter no reconocido, generan un **error léxico** y el análisis continúa registrando el resto. Cada token reconocido y cada error se agregan a listas estáticas (TokenInfo, Error) que se limpian al inicio de cada análisis (Lexer.clearErrors()).

5. Analizador sintáctico (Parser.cup)

Gramática **LALR** generada con CUP; símbolo inicial `programa`. Precedencia (de menor a mayor): `|| < &&` < comparaciones (`== != >= <= > <`, `&& || !`); < < paréntesis/ funciones/literales así se cumple el orden pedido en el enunciado.

Cada regla construye directamente los objetos del modelo mientras se reduce la entrada: una estrategia produce un `Strategy`; una partida produce un `Match`; cada `if/else` produce un `IfRule/ElseRule`; las expresiones arman un árbol de `Expression`. Al reducir `programa`, todo se guarda en el *singleton* `ProgramStore`.

El parser sobrescribe `report_error(...)`, `syntax_error(...)` y `unrecovered_syntax_error(...)` de CUP para traducir cada fallo en un `Error` tipo "Sintáctico" con el lexema, tipo de token y posición. No se implementa recuperación de errores: el análisis se detiene en el primer error sintáctico, tal como pide el enunciado.

6. Modelo del lenguaje (battlescript.model)

Clase	Contenido
Action (enum)	Las 10 acciones fijas, con clase dueña, poder, costo, prioridad y ofensividad.
ClassType (enum)	MAGE, WARRIOR.
Strategy	Nombre, clase, acción inicial, lista de Rule.
Match	Nombre, dos jugadores, rondas, Scoring, Bonuses.
Scoring / Bonuses	Valores de puntuación y de combos/bonos.
RunInstruction / MainBlock	Partidas a ejecutar y semilla de cada run.
ProgramStore	<i>Singleton</i> con el resultado completo del análisis sintáctico.

7. Expresiones, reglas y contexto

`Expression` es la interfaz base evaluada contra un `Context`. `BinaryExpression` implementa los operadores relacionales y lógicos, con **evaluación de corto circuito** en `&&/||`. `UnaryExpression` implementa `!`. `ConstantExpression` y `VariableExpression` resuelven literales y estados del sistema. `Function-Call` implementa `get_move`, `last_move`, `get_moves_count` y `get_last_n_moves`, validando índices y historiales vacíos. `IfRule` retorna su acción solo si la condición es verdadera; `ElseRule` siempre la retorna.

Antes de evaluar las reglas de un jugador en cada ronda, `BattleEngine` arma su `Context` (salud, recurso, score, historial propio y del rival, `round_number`, `total_rounds` y `random`). El valor `random` se genera **una vez por combatiente y por ronda**, incluso si la estrategia no lo usa, para que todas sus referencias dentro de esa ronda coincidan.

8. Motor de batalla (`engine.battle`)

`Fighter` inicializa las estadísticas según la clase:

Estadística	Mago	Guerrero
Vida máxima	100	140
Recurso máximo	120	100
Ataque físico	5	22
Poder mágico	25	0
Armadura	8	20
Resistencia mágica	18	8
Velocidad	14	10

`BattleEngine.run(match, p1, p2, seed)`: crea dos `Fighter` y dos `Random` (`seed` y `seed+1`). En cada ronda arma el contexto de ambos, selecciona su acción (recorriendo las reglas en orden; si ninguna aplica, usa la acción inicial), determina quién actúa primero por **prioridad de acción** y, en empate, por **velocidad** (y si también empatan, el primero declarado en `players`), ejecuta ambas acciones, reinicia los efectos temporales (defensa, `WAR_CRY`) y revisa combos comparando los últimos 3 movimientos del historial con `bonuses`.

`executeAction(...)`: si falta recurso, la acción no surte efecto, no entra al historial y se resta `failed_action_penalty` (sin bajar de cero). Ofensiva → calcula daño; si el rival defiende, el daño se reduce a la mitad y el defensor gana `successful_defense`; el atacante gana `daño × damage_point`. Defensiva (`MAGIC_BARRIER/SHIELD_BLOCK`) → activa “defendiendo” por la ronda. Curación (`HEALING_RUNE`) → cura sin superar el máximo; gana `vida curada × healing_point`. Recuperación (`MEDITATE/REST`) → recupera recurso. `WAR_CRY` → +10 de ataque al próximo golpe. Si el rival queda en 0 de vida, se otorga `victory_bonus`.

Fórmulas de daño:

Daño físico = `poder_acción + ataque_físico + bono_WAR_CRY - armadura_defensor`
Daño mágico = `poder_acción + poder_mágico - resistencia_mágica_defensor`
`dañoFinal` = `max(1, daño)`
`dañoDefendido` = `floor(dañoFinal × 0.50)` (si el defensor está defendiendo)

Prioridad (mayor primero): `MAGIC_BARRIER/SHIELD_BLOCK` (7) > `WAR_CRY` (6) > `HEALING_RUNE` (5) > `ARCANE_BOLT/SLASH` (4) > `FIREBALL/HEAVY_STRIKE` (2) > `MEDITATE/REST` (1).

Ganador (`determineWinner`): primero por muerte de un combatiente (ambos muertos → empate); si ambos sobreviven al límite de rondas, por mayor puntuación → mayor vida → mayor recurso → empate.

`ProgramExecutor` recorre las `RunInstruction` del `main` en orden y, por cada partida, llama a

`BattleEngine.run(...)` con la semilla indicada.

9. Análisis y reportes (`engine.analysis`)

`SourceAnalyzer` orquesta `lexer` + `parser` y, si no hubo errores léxicos/sintácticos, ejecuta `SemanticValidator`, que valida lo que la gramática no puede expresar por sí sola: nombres únicos de estrategias/partidas, existencia de `main`, jugadores referenciados válidos, `rounds > 0`, valores de `scoring` no negativos, acciones de combo/clase correspondientes (`mage`↔acciones de mago, `warrior`↔acciones de guerrero) y `seed > 0` en cada `run`. Esta validación es intencionalmente sencilla, ya que el proyecto no requiere un análisis semántico extenso.

`ReportGenerator` arma un HTML autocontenido con tabla de tokens, tabla de errores (o mensaje de éxito) y pie con fecha/hora, y lo guarda en `reports/<nombre>_<fecha_hora>.html` (carpeta creada automáticamente), escapando el HTML para evitar romper el marcado.

10. Interfaz gráfica

Interfaz (Swing) expone un `JTextArea` para el código, dos `JTable` en un `JTabbedPane` ("Tokens"/"Errores"), un área de resultados, una etiqueta de estado, botones (Nuevo, Abrir, Guardar, Ejecutar) y menús Archivo/Ayuda. `InterfazController` conecta la vista con el motor: implementa `createNew()`, `open()`, `save()` y `run()`, usa `JFileChooser` filtrado a `*.btl` y `JOptionPane` para avisos y el resumen final de partidas.

11. Compilación y ejecución

Proyecto Ant/NetBeans. El target `-pre-compile` depende de `generate`, que ejecuta la tarea `jflex` sobre `Lexer.jflex` y la tarea `cup` sobre `Parser.cup` (`parser="Parser"`, `interface="true"`, `expect="6"`), generando `Lexer.java`, `Parser.java` y `sym.java` antes de compilar. `-post-clean` elimina esos archivos generados al limpiar.

12. Mantenimiento futuro

- **Nueva acción:** agregarla al enum `Action`, su token en `Lexer.jflex` y, si aplica, actualizar prioridades/fórmulas en `BattleEngine`.
- **Nuevo estado del sistema:** token en `Lexer.jflex`, no terminal en `Parser.cup` (`expresion_primaria`), campo en `Context`, resolución en `VariableExpression`.
- **Nueva función del sistema:** token, regla función en `Parser.cup`, implementación en `FunctionCall`.
- **Reglas de puntuación/victoria:** cambios en `BattleEngine.executeAction(...)` y `determineWinner(...)`.
- **Formato del reporte:** editar `ReportGenerator.buildHtml(...)`.